11 — Multi-seesion cohort and reliability QC

This notebook extends the current single-session analysis into a multi-session cohort.

Publication safety boundary

This notebook is intended for publication analysis without overwriting long-running artifacts. By default, reruns write derived manuscript outputs to a versioned namespace such as `reports/tables/publication_upgrade_v2` and `reports/figures/publication_upgrade_v2`. Existing `data/`, `models/`, and canonical `reports/` artifacts should be treated as protected evidence unless an explicit regeneration plan is chosen.

```
from pathlib import Path
import os
import sys
import yaml
import numpy as np
import pandas as pd

PROJECT_ROOT = Path(
    r"C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-natural-mov
).resolve()
SRC_DIR = PROJECT_ROOT / "src"
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))
os.chdir(PROJECT_ROOT)

cfg_path = PROJECT_ROOT / "configs" / "publication_upgrade.yaml"
if not cfg_path.exists():
    raise FileNotFoundError(f"Could not find publication config: {cfg_path}")
with cfg_path.open("r", encoding="utf-8") as f:
    pub_cfg = yaml.safe_load(f)
cfg = pub_cfg

PUBLICATION_RUN_LABEL = os.environ.get("PUBLICATION_RUN_LABEL", "publication_u
ALLOW_CANONICAL_PUBLICATION_WRITE = os.environ.get("ALLOW_CANONICAL_PUBLICATIO
if PUBLICATION_RUN_LABEL == "publication_upgrade" and not ALLOW_CANONICAL_PUBL
    raise RuntimeError(
        "Refusing to write into the canonical publication_upgrade namespace. "
        "Use PUBLICATION_RUN_LABEL=publication_upgrade_v2 or set "
        "ALLOW_CANONICAL_PUBLICATION_WRITE=1 intentionally."
    )
```

```
from v1_manifold_publication.paths import PublicationPaths, ensure_publicatior
from v1_manifold_publication.readiness import build_readiness_report, format_r

paths = PublicationPaths.from_config(cfg_path)
paths.publication_tables_dir = paths.root / "reports" / "tables" / PUBLICATION
paths.publication_figures_dir = paths.root / "reports" / "figures" / PUBLICATI
paths.versioned_processed_dir = paths.processed_dir / PUBLICATION_RUN_LABEL
ensure_publication_dirs(paths)

print("Using PROJECT_ROOT:", PROJECT_ROOT)
print("Publication run label:", PUBLICATION_RUN_LABEL)
print("Publication tables:", paths.publication_tables_dir)
print("Publication figures:", paths.publication_figures_dir)
print("Versioned processed feature dir:", paths.versioned_processed_dir)
print("Read-only readiness tier:", build_readiness_report(PROJECT_ROOT)["tier"
```

```
Using PROJECT_ROOT: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication run label: publication_upgrade_v2
Publication tables: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold
Publication figures: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifol
Versioned processed feature dir: C:\Users\Peter\Documents\projects\NeuroAI\la
Read-only readiness tier: exploratory_or_scaffold
```

```
from v1_manifold_publication.pipeline import validate_environment
from v1_manifold_publication.cohort import discover_existing_session_files

env = validate_environment(PROJECT_ROOT)
save_table(env, paths.publication_tables_dir / "11_environment_validation.cs
display(env)

print("Tables directory:", paths.tables_dir)
print("Processed directory:", paths.processed_dir)
print("Interim directory:", paths.interim_dir)
print("Publication tables directory:", paths.publication_tables_dir)

print("\nNumber of report tables:", len(list(paths.tables_dir.glob('*.csv'))
print("Number of processed embeddings:", len(list(paths.processed_dir.glob('
print("Number of interim tensors:", len(list(paths.interim_dir.glob('session

session_index = discover_existing_session_files(paths.processed_dir, paths.i
save_table(session_index, paths.publication_tables_dir / "existing_session_f
display(session_index)

print(format_readiness_markdown(build_readiness_report(PROJECT_ROOT)))
```

	path	exists	type
0	notebooks	True	dir

1	src	True	dir
2	data	True	dir
3	reports	True	dir
4	reports/tables	True	dir
5	reports/figures	True	dir
6	core_python_dependencies	True	import

```
Tables directory: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v
Processed directory: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifol
Interim directory: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-
Publication tables directory: C:\Users\Peter\Documents\projects\NeuroAI\laten

Number of report tables: 56
Number of processed embeddings: 1
Number of interim tensors: 1
```

	session_id	embedding_file	
0	500855614	C:\Users\Peter\Documents\projects\NeuroAI\late...	\Users\Peter\Documents\p

```
# Publication Readiness Audit

Project root: `C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-n
Current tier: `exploratory_or_scaffold`

## Inventory
- `raw_nwb_files`: 4
- `interim_tensors`: 1
- `processed_embeddings`: 1
- `processed_feature_tables`: 3
- `model_files`: 186
- `report_tables`: 56
- `publication_tables`: 17
- `figures`: 66

## Cohort
- Processed sessions: 1 (500855614)
- Cohort sessions: 3 (500855614, 500964514, 501271265)
- Layers: L2/3
- Cre lines: Cux2-CreERT2

## Publication Tables
- `11_publication_candidate_cohort.csv`: ok, rows=3, columns=15
- `11_noise_ceiling_summary.csv`: ok, rows=1, columns=4
- `13_latent_feature_decoding_blockcv_summary.csv`: ok, rows=25, columns=29
- `13_stimulus_to_population_encoding_summary.csv`: ok, rows=1, columns=35
- `14_geometry_with_session_metadata.csv`: ok, rows=5, columns=27
- `15_brain_model_alignment_analytic_features.csv`: ok, rows=5, columns=7
- `15_brain_model_alignment_deep_features.csv`: missing_or_empty, rows=0, col
- `16_pairwise_cross_session_manifold_alignment.csv`: missing_or_empty, rows=
```

```
## Blocking Issues
- Only 1 processed session(s); target at least 12.
- Only 1 represented layer(s); target at least 3.
- Cross-session manifold alignment table is empty or missing.
- Deep model alignment table is empty or missing.
```

```
from v1_manifold_publication.pipeline import build_existing_session_index
from v1_manifold_publication.cohort import select_publication_cohort, summar


# ---------------------------------------------------------------------
# Resolve directories safely
# ---------------------------------------------------------------------
try:
    project_root = Path(PROJECT_ROOT)
except NameError:
    project_root = Path.cwd()

external_dir = project_root / "data" / "external"
reports_tables_dir = getattr(paths, "tables_dir", project_root / "reports" /
publication_tables_dir = getattr(
    paths,
    "publication_tables_dir",
    reports_tables_dir / "publication_upgrade",
)

publication_tables_dir.mkdir(parents=True, exist_ok=True)

print("Project root:", project_root)
print("External metadata directory:", external_dir)
print("Reports tables directory:", reports_tables_dir)
print("Publication tables directory:", publication_tables_dir)


# ---------------------------------------------------------------------
# Helper functions
# ---------------------------------------------------------------------
def _read_candidate_table(path):
    try:
        df = pd.read_csv(path)
        df["source_table"] = path.name
        return df
    except Exception as exc:
        print(f"Skipping unreadable table: {path.name} | {exc}")
        return None


def _has_session_identifier(df):
    possible_id_cols = [
```

```
        "session_id",
        "ophys_session_id",
        "ophys_experiment_id",
        "experiment_id",
        "id",
    ]
    return any(col in df.columns for col in possible_id_cols)


def _standardize_session_column(df):
    df = df.copy()

    if "session_id" not in df.columns:
        for candidate in [
            "ophys_session_id",
            "ophys_experiment_id",
            "experiment_id",
            "id",
        ]:
            if candidate in df.columns:
                df["session_id"] = df[candidate]
                break

    return df


# ---------------------------------------------------------------------
# Collect candidate metadata tables
# ---------------------------------------------------------------------
candidate_paths = []

if external_dir.exists():
    candidate_paths += list(external_dir.glob("*experiment*.csv"))
    candidate_paths += list(external_dir.glob("*metadata*.csv"))
    candidate_paths += list(external_dir.glob("*.csv"))

if reports_tables_dir.exists():
    candidate_paths += list(reports_tables_dir.glob("01_*experiment*.csv"))
    candidate_paths += list(reports_tables_dir.glob("01_*metadata*.csv"))
    candidate_paths += list(reports_tables_dir.glob("01_*.csv"))

# Remove duplicates while preserving order.
seen = set()
candidate_paths_unique = []
for path in candidate_paths:
    path = Path(path)
    if path not in seen:
        candidate_paths_unique.append(path)
        seen.add(path)

experiment_level_tables = []
```

```
experiment_level_tables = []
summary_only_tables = []

for path in candidate_paths_unique:
    df = _read_candidate_table(path)

    if df is None or df.empty:
        continue

    if _has_session_identifier(df):
        experiment_level_tables.append((path, _standardize_session_column(df
    else:
        summary_only_tables.append((path, df))

print("\nCandidate metadata tables inspected:")
if candidate_paths_unique:
    for path in candidate_paths_unique:
        print(" -", path.name)
else:
    print(" - None found")


# ----------------------------------------------------------------------
# Use experiment-level metadata if available; otherwise fallback
# ----------------------------------------------------------------------
if experiment_level_tables:
    selected_path, experiments = experiment_level_tables[0]

    print(f"\nUsing experiment-level metadata table: {selected_path}")
    print("Columns:", list(experiments.columns))

    cohort = select_publication_cohort(experiments, cfg)
    cohort_summary = summarize_cohort(cohort)

    save_table(
        cohort,
        publication_tables_dir / "11_publication_candidate_cohort.csv",
    )
    save_table(
        cohort_summary,
        publication_tables_dir / "11_publication_candidate_cohort_summary.cs
    )

    display(cohort.head())
    display(cohort_summary)

else:
    print("\nNo experiment-level metadata table with a session identifier wa
    print("Falling back to processed sessions already present in this projec

    session_index = build_existing_session_index(
```

```
            project_root / "configs" / "publication_upgrade.yaml"
        )

        if session_index.empty:
            raise FileNotFoundError(
                "No experiment-level metadata table was found, and no processed
                "could be reconstructed from existing embedding/tensor files."
            )

        cohort = session_index.copy()
        cohort["cohort_source"] = "processed_session_index_fallback"
        cohort["publication_status"] = "already_processed_single_session"

        cohort_summary = pd.DataFrame(
            {
                "metric": [
                    "n_processed_sessions",
                    "n_embedding_files",
                    "n_tensor_files",
                    "cohort_source",
                ],
                "value": [
                    cohort["session_id"].nunique(),
                    cohort["embedding_file"].notna().sum(),
                    cohort["tensor_file"].notna().sum(),
                    "processed_session_index_fallback",
                ],
            }
        )

        save_table(
            cohort,
            publication_tables_dir / "11_publication_candidate_cohort.csv",
        )
        save_table(
            cohort_summary,
            publication_tables_dir / "11_publication_candidate_cohort_summary.cs
        )

        display(cohort)
        display(cohort_summary)

    if summary_only_tables:
        print("\nSummary-only tables were found but not used as experiment-level
        for path, df in summary_only_tables[:10]:
            print(f" - {path.name}: columns={list(df.columns)}")
```

```
Project root: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na
External metadata directory: C:\Users\Peter\Documents\projects\NeuroAI\latent
Reports tables directory: C:\Users\Peter\Documents\projects\NeuroAI\latent-ma
Publication tables directory: C:\Users\Peter\Documents\projects\NeuroAI\laten
```

```
Candidate metadata tables inspected:
 - allen_v1_natural_movie_experiments.csv
 - 01_selected_experiments_by_cre_depth_layer.csv
 - 01_selected_experiments_by_cre_line.csv
 - 01_available_v1_natural_movie_cre_lines.csv

Using experiment-level metadata table: C:\Users\Peter\Documents\projects\Neur
Columns: ['id', 'imaging_depth', 'targeted_structure', 'cre_line', 'reporter_
```

	id	imaging_depth	targeted_structure	cre_line	reporter_line	acq
0	500855614	175	VISp	Cux2-CreERT2	Ai93(TITL-GCaMP6f)	
1	500964514	175	VISp	Cux2-CreERT2	Ai93(TITL-GCaMP6f)	
2	501271265	175	VISp	Cux2-CreERT2	Ai93(TITL-GCaMP6f)	

	cre_line	putative_layer	n_sessions	n_mice	median_cells
0	Cux2-CreERT2	L2/3	3	3	3

```
Summary-only tables were found but not used as experiment-level cohort tables
 - 01_selected_experiments_by_cre_depth_layer.csv: columns=['cre_line', 'imag
 - 01_selected_experiments_by_cre_line.csv: columns=['cre_line', 'n_experimen
 - 01_available_v1_natural_movie_cre_lines.csv: columns=['cre_line', 'imaging
```

```
# ----------------------------------------------------------------------
# Repeat reliability and noise-ceiling analysis
# ----------------------------------------------------------------------
# This does not retrain any decoder or embedding model.
# The core project stores tensors as [repeat, cell, frame], so the reliabili
# helper is called with an explicit layout to avoid silently swapping cells

from v1_manifold.preprocessing import load_trial_tensor_h5
from v1_manifold_publication.reliability import (
    split_half_reliability,
    noise_ceiling_from_repeats,
)

publication_tables_dir = paths.publication_tables_dir
publication_tables_dir.mkdir(parents=True, exist_ok=True)

tensor_files = sorted(paths.interim_dir.glob("session_*_tensor.h5"))
if not tensor_files:
    raise FileNotFoundError(f"No tensor files found in {paths.interim_dir}.
```

```
rows = []
for tensor_path in tensor_files:
    session_id = tensor_path.name.split("_")[1]
    print(f"Computing repeat reliability for session {session_id}")
    print(f"Tensor file: {tensor_path}")

    try:
        tensor, R = load_trial_tensor_h5(tensor_path)
        expected_frames = int(R.shape[0])
        print("Tensor shape [repeat, cell, frame]:", tensor.shape)
        print("Population response shape [frame, cell]:", R.shape)

        rel = split_half_reliability(
            tensor,
            n_splits=100,
            rng=cfg["project"]["random_seed"],
            layout="repeats_cells_frames",
            expected_frames=expected_frames,
        )
        rel.insert(0, "session_id", session_id)
        rel.insert(1, "tensor_layout", "repeat_cell_frame")
        save_table(rel, publication_tables_dir / f"11_repeat_reliability_ses

        ceiling = noise_ceiling_from_repeats(
            tensor,
            layout="repeats_cells_frames",
            expected_frames=expected_frames,
        )
        rows.append({
            "session_id": session_id,
            "tensor_file": str(tensor_path),
            "tensor_layout": "repeat_cell_frame",
            "expected_frames": expected_frames,
            **ceiling,
        })
    except Exception as exc:
        rows.append({"session_id": session_id, "tensor_file": str(tensor_pat

reliability_summary = pd.DataFrame(rows)
save_table(reliability_summary, publication_tables_dir / "11_noise_ceiling_s
display(reliability_summary)
```

```
Computing repeat reliability for session 500855614
Tensor file: C:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-nat
Tensor shape [repeat, cell, frame]: (10, 163, 900)
Population response shape [frame, cell]: (900, 163)
```

	session_id	tensor_file	tensor_layout	expe
0	500855614	C:	repeat_cell_frame	

0	500855614	\Users\Peter\Documents\projects\NeuroAI\late...	repeat_cell_frame

Multi-session run plan

This cell produces versioned publication result tables when the notebook is run in the full scientific environment.

```
# Publication result cell: multi-session run plan and manuscript claim gates
# This writes only into the versioned publication table namespace.
from v1_manifold_publication.cohort import discover_existing_session_files,
from v1_manifold_publication.multisession import build_session_run_plan, cla
from v1_manifold_publication.readiness import build_readiness_report

metadata_path = paths.root / "data" / "external" / "allen_v1_natural_movie_e
existing = discover_existing_session_files(paths.processed_dir, paths.interi

if metadata_path.exists():
    experiments = pd.read_csv(metadata_path)
    cohort_for_plan = select_publication_cohort(experiments, cfg)
else:
    cohort_for_plan = existing.copy()

run_plan = build_session_run_plan(cohort_for_plan, existing)
save_table(run_plan, paths.publication_tables_dir / "11_publication_multises

gates = claim_gate_table(build_readiness_report(PROJECT_ROOT))
save_table(gates, paths.publication_tables_dir / "11_publication_claim_gates

display(run_plan)
display(gates)
```

	id	imaging_depth	targeted_structure	cre_line	reporter_line	acq
0	500855614	175	VISp	Cux2-CreERT2	Ai93(TITL-GCaMP6f)	
1	500964514	175	VISp	Cux2-CreERT2	Ai93(TITL-GCaMP6f)	
2	501271265	175	VISp	Cux2-CreERT2	Ai93(TITL-GCaMP6f)	

3 rows × 21 columns

	claim	minimum_evidence	status
	single-session V1 natural-movie	one processed session with real features	

0	single-session V1 natural-movie decoding	one processed session with real features and h...	supported
1	multi-session replication	at least 12 processed sessions across mice	not_ready
2	layer-specific manifold geometry	at least 3 layers with cell-count-matched sess...	not_ready
3	deep-model brain alignment	nonempty DNN/ViT alignment table with random a...	not_ready